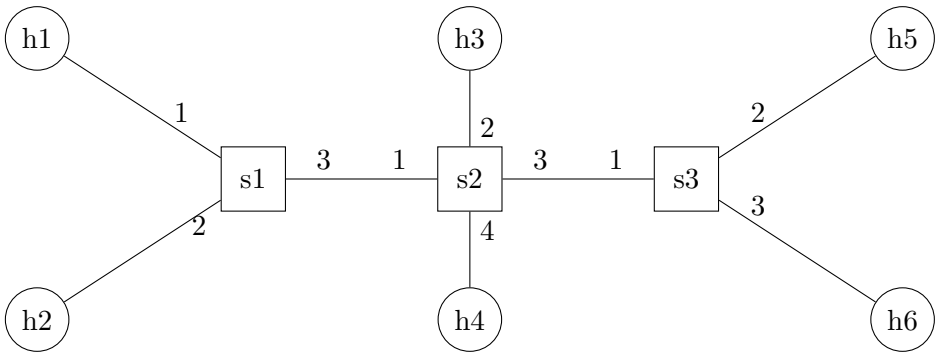
Software Defined Networking – 11 settembre 2018

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Nelle risposte usare per ogni pacchetto il seguente formato:

Nodo → interfaccia: MAC sorgente → MAC destinazione; ethertype; IP sorgente → IP destinazione; TTL; protocollo; porta sorgente → porta destinazione;

Negli switch sono preinstallate le seguenti regole.

Switch s1

Priority	Match	Action
0	*	Output(CONTROLLER,128)
10	eth_type = ETH_TYPE_IP ip_proto = IPPROTO_UDP	Output(CONTROLLER, OFPCML_NO_BUFFER)
5	eth_type = ETH_TYPE_IP ipv4_dst = IP3	PushMPLS() SetField(mpls_label=1003) Output(3)
5	eth_type = ETH_TYPE_MPLS mpls_label = 1001	PopMPLS() Output(1)

Switch s2		
Priority	Match	Action
0	*	Output(CONTROLLER,128)
10	eth_type = ETH_TYPE_IP ip_proto = IPPROTO_UDP	Output(CONTROLLER, OFPCML_NO_BUFFER)
5	eth_type = ETH_TYPE_IP ipv4_dst = IP1	PushMPLS() SetField(mpls_label=1001) Output(1)
5	eth_type = ETH_TYPE_MPLS mpls_label = 1003	PopMPLS() Output(2)

Switch s3		
Priority	Match	Action
0	*	Output(CONTROLLER,128)
10	eth_type = ETH_TYPE_IP ip_proto = IPPROTO_UDP	Output(CONTROLLER, OFPCML_NO_BUFFER)

L'host h1 porta 1234 apre una connessione TCP verso h3 porta 80.

(a) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1.

(b) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

(c) L'host h3 risponde. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

L'host h1 porta 3322 invia un datagramma UDP verso h3 porta 123.

(d) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1.

(e) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

(f) L'host h3 risponde. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

L'host h4 porta 7777 apre una connessione TCP verso l'host h6 porta 245.

(g) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

L'host h4 porta 7777 apre una connessione TCP verso l'host h5 porta 245.

(h) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

(i) Scrivere il contenuto finale delle tabelle openflow.

Programma 1 – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
    parser = datapath.ofproto_parser; in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
    buffer_id = msg.buffer_id; data = msg.data

    if (pkt_udp is not None):
        pkt_ipv4.ttl = pkt_ipv4.ttl - 2
        if ( pkt_udp.dst_port & 2 == 1)
            pkt_udp.dst_port = pkt_udp.dst_port + 1
        pkt.serialize()

        actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
        out = parser.OFPPacketOut(datapath=datapath, data = pkt.data
            in_port=in_port, actions=actions)
        datapath.send_msg(out)
        return

    if ( pkt_tcp is not None and datapath.id == 2 and pkt_ipv4.dst in [ IP5, IP6 ] ):
        actions = [parser.OFPActionOutput(3)]
        match = parser.OFPMatch(eth_type=ETH_TYPE_IP, ipv4_dst= pkt_ipv4.dst,
            ip_proto=IPPROTO_TCP, tcp_dst=pkt_tcp.dst_port)
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3,
            match=match, instructions=inst, buffer_id=buffer_id)
        datapath.send_msg(mod)
        return

    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
    out = parser.OFPPacketOut(datapath=datapath, buffer_id=buffer_id,
        in_port=in_port, actions=actions)
    datapath.send_msg(out)
```

Esercizio 2

Un sistema di blocco di URL fraudolente usa un filtro di Bloom costituito da un array di 2^{12} bit e 3 funzioni hash.

(a) Calcolare la probabilità di falso positivo dopo aver effettuato l'inserimento di 1 000 URL.

(b) Calcolare la dimensione minima dell'array per avere un probabilità di falso positivo di 10^{-6} dopo 1 000 inserimenti.

Si consideri la famiglia di funzioni hash $h_i(x) = i(x+1) \bmod 2^{12}$ per $1 \leq i \leq 3$

(c) Mostrare lo stato del filtro dopo l'inserimento dell'elemento 34.

(d) Verificare l'appartenza al dataset dell'elemento 231.

Esercizio 3

Si consideri un'interfaccia gestita da uno scheduler Deficit Round Robin. La velocità di trasmissione è 8 Gbit/s. Al tempo $t = 0$ sono presenti i seguenti pacchetti, di cui viene indicata la lunghezza in byte, ordinati dal più recente al più vecchio:

Coda 1:	50, 150, 150
Coda 2:	225, 50, 50
Coda 3:	100, 150, 100
Coda 4:	100

(a) Si considerino quanti di lunghezza 100. Simulare l'algoritmo DRR, indicando il tempo al quale termina ogni trasmissione ed il deficit residuo.

(b) Ripetere la simulazione considerando che: le code 1 e 3 hanno quanti di lunghezza 100; le code 2 e 4 hanno quanti di lunghezza 50; al tempo $t = 600$ ns si presenta alla coda 4 un pacchetto di lunghezza 100.

(c) Si consideri uno scenario con lunghezza massima dei pacchetti 1000 byte. Calcolare il valore minimo del quanto che garantisca che: ad ogni iterazione si trasmetta almeno un pacchetto per ogni coda; la coda 2 abbia banda doppia rispetto alle code 3 e 4; le code 2, 3 e 4 abbiano collettivamente la stessa banda della coda 1.

Esercizio 4

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Source Address	Destination Address	Source Port	Destination Port	Protocol
32.0.0.0/5	44.0.0.0/6	123	*	TCP
44.0.0.0/6	248.0.0.0/5	*	22	TCP
64.0.0.0/3	32.0.0.0/5	*	53	UDP
*	96.0.0.0/3	*	123	UDP
248.0.0.0/5	64.0.0.0/3	123	*	UDP
96.0.0.0/3	*	*	80	*

(a) Definire il bitvector set per ciascun campo.

(b) L'algoritmo di ricerca del prefisso IP sorgente (prima colonna) è implementato usando un multibit trie a passo 3. Disegnare il trie.

(c) L'algoritmo di ricerca del prefisso IP destinazione (seconda colonna) è implementato usando un multibit trie a passo variabile. Ad ogni nodo, scegliere il passo in modo da minimizzare il numero di figli. Disegnare il trie.

(d) Si consideri il pacchetto (64.10.0.1, 96.0.0.2, 80, 123, UDP). Applicare la procedura di BVLS per identificare gli eventuali match.